Application Protocols, Framing and Reliability over UDP

Lecture 14

Computer Networks and Distributed Programming

Where we are: the protocol you never wrote

Five versions of the same chat, and every one of them changed who runs the code — a process, a thread, one loop, a pool.

- What travels on the connection never changed: a line of text, with no name, no length and no way to be wrong.
- The server cannot refuse anything, because it has no way to answer anything except with more text.
- And all of it works because TCP is underneath: bytes arrive, in order, exactly once, or the connection dies.
- Today the protocol becomes an object you design: message types, boundaries, error codes.
- Then we take TCP away and put datagrams underneath, and write by hand what it was doing for us.

The four questions, asked of your own protocol

Already defined

An application-layer protocol fixes the types of message, their syntax, the semantics of the fields, and the rules of exchange. You have used three of them by now.

- Types: a client that joins, sends, leaves; a server that delivers, confirms, refuses. Yours has one, and it is unnamed.
- Syntax: where each message begins and ends, and how the fields inside it are separated. This is the hard one.
- Semantics: what a name means, what an empty body means, what the server does with a message for somebody who left.
- Rules: who speaks first, what may be sent before joining, and what happens to a message that arrives out of turn.

Text or bytes

Text	Binary
you can type it with `nc`	you need a program to send anything
any length: a number is 1 to 10 digits	fixed width: a number is always 4 bytes
parsing is scanning and converting	parsing is copying and byte-swapping
HTTP, SMTP, WHOIS	DNS, TLS, everything below transport

- Text costs bytes and parsing; binary costs every debugging session you will ever have on it.
- Neither of them answers the question of where a message ends: that is a separate decision, and it comes next.
- A common compromise: a binary header that says how long the message is, and a body that is whatever you like.

Encoding a message as text

```c
char msg[128];
size_t off = 0;
for (int i = 0; i < n; i++) {                          /* encode */
    int k = snprintf(msg + off, sizeof msg - off, "%d ", a[i]);
    if (k < 0 || (size_t) k >= sizeof msg - off) return -1;   /* would not fit */
    off += (size_t) k;
}
msg[off++] = '\n';                        /* the delimiter is part of the protocol */
send_all(fd, msg, off);

char *save, *tok = strtok_r(line, " ", &save);          /* decode */
for (i = 0; tok != NULL && i < MAX_VALUES; i++) {
    vals[i] = atoi(tok);
    tok = strtok_r(NULL, " ", &save);
}
```

- `strtok` keeps its state in a static: two threads parsing two messages destroy each other's scan.
- `atoi` cannot fail: `"abc"` is zero. To reject bad input you need `strtol`.

Encoding it as bytes

```c
uint32_t n_net = htonl(n);                          /* sender: the count first */
send_all(fd, &n_net, sizeof n_net);
for (uint32_t i = 0; i < n; i++) {
    uint32_t v_net = htonl(vals[i]);
    send_all(fd, &v_net, sizeof v_net);
}

uint32_t n_net, n;                                  /* receiver: the count decides */
if (recv_all(fd, &n_net, sizeof n_net) != sizeof n_net) return -1;
n = ntohl(n_net);
if (n > MAX_VALUES) return -1;          /* never allocate what the peer asks for */
for (uint32_t i = 0; i < n; i++) { ... }
```

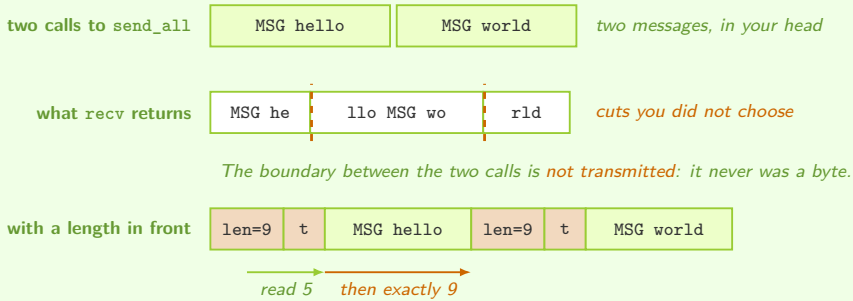
- `uint32_t` from `<stdint.h>`, not `int`: the width of `int` is a property of the compiler, not of the protocol.
- Network byte order applies to your data too, not only to the ports and addresses in the headers.
- The count sent first is the framing: the receiver knows how much follows before it reads it.

Three ways to mark the end of a message

You have met the question twice — with two lines that arrived as one read, and with an HTTP reply that never ended. There are only three answers.

- A delimiter: scan the incoming bytes for it. Cheap to type by hand, and the payload must never contain it, or you must escape it.
- A length in front: read a fixed-size header, then exactly that many bytes. No scanning, no escaping, and binary payloads are free.
- Closing the connection: the end of the stream is the end of the message. Works once, and only for the last one.
- HTTP uses the first for its header and the second for its body; `whois` uses the third; DNS over TCP uses the second.
- Your protocol has to pick one, and the receiver must be able to apply it without guessing.

From a stream to messages



The stream is the same in both halves. What changes is that in the second the boundaries are bytes you sent, so the receiver can find them.

Reading exactly *n* bytes

```c
ssize_t recv_all(int fd, void *buf, size_t n) {
    size_t got = 0;
    char *p = buf;
    while (got < n) {
        ssize_t r = recv(fd, p + got, n - got, 0);
        if (r > 0) { got += (size_t) r; continue; }
        if (r == 0) return (ssize_t) got;   /* peer closed: short, not an error */
        if (errno == EINTR) continue;       /* a signal ran: not an error either */
        return -1;
    }
    return (ssize_t) n;
}
```

- A length in front is useless without this: `recv` may return fewer bytes than you asked for, and usually will.
- These functions have names: `readn`, `writen`, `readline` of *UNIX Network Programming*. You will find them in network code everywhere, and they are not in libc.

Writing exactly *n* bytes

```c
ssize_t send_all(int fd, const void *buf, size_t n) {
    size_t sent = 0;
    const char *p = buf;
    while (sent < n) {
        ssize_t w = send(fd, p + sent, n - sent, 0);
        if (w > 0) { sent += (size_t) w; continue; }
        if (errno == EINTR) continue;
        return -1;                          /* the peer is gone, or worse */
    }
    return (ssize_t) n;
}
```

- `send` does not transmit: it copies into the socket's send buffer in the kernel, whose size is `SO_SNDBUF`.
- If that buffer is full of data not yet acknowledged, it copies less than you asked — a short write — and returns how much.
- On a blocking socket it may also wait; on a non-blocking one it returns `EAGAIN`, which the event loop already knows about.

Reading one frame

```c
int read_frame(int fd, uint8_t *type, char *body, size_t max) {
    uint8_t h[5];             /* never send a struct: padding is the compiler's */
    if (recv_all(fd, h, sizeof h) != (ssize_t) sizeof h) return -1;

    uint32_t len;
    memcpy(&len, h, 4);
    len = ntohl(len);
    *type = h[4];

    if (len > max) return -1;               /* refuse before allocating */
    if (recv_all(fd, body, len) != (ssize_t) len) return -1;
    return (int) len;
}
```

- Two reads, always: the header has a fixed size, so the first one is the same every time; the second one is what it says.
- The wire format is a sequence of bytes, not a `struct`: padding, alignment and field order differ between machines.

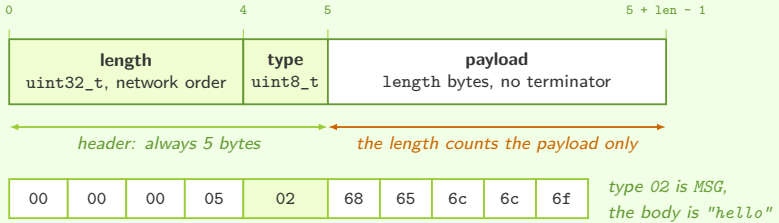
The length you believed

A client announces a body of four gigabytes and then sends nothing. What does your server do?

It calls `malloc` with a number the peer chose, and then waits inside `recv_all` for bytes that will never come — holding the memory, and a worker, for as long as the client feels like it.

- Every field that comes from the network is an input, and a length is the most dangerous of them.
- So the protocol declares a maximum, and the check happens before the allocation and before the read.
- The client that hangs is answered by the other tool of this lecture: a read timeout on the socket.

The chat protocol, written down



- From the client: `JOIN` with a name, `MSG` with a line, `QUIT` with an empty body.
- From the server: `OK`, `ERR` with a code, and `DELIVER` with the sender's name and the line.

Saying no

A protocol that cannot refuse has only one way to disagree with a client: hanging up. That is why HTTP answers with a number and a reason.

- The code is for the program: it decides whether to retry, give up, or ask the user something.
- The string is for the person: nobody debugs a protocol by looking up `ERR 7` in your source.
- Group the codes: what the client got wrong, what the server could not do, what is temporary and worth retrying.
- An error is a normal reply: the connection stays open, and the next message is served as usual.
- Closing is for what you cannot answer at all — a header that does not parse, a length you refuse, a peer that stopped talking.

A blocking call interrupted by a signal

```c
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd >= 0) break;
    if (errno == EINTR) continue;   /* a handler ran: nothing went wrong */
    perror("accept");
    continue;
}
```

- Any blocking call — `accept`, `read`, `recv`, `select` — can return -1 with `EINTR` when a handler runs while it waits.
- In your server there is one: the handler that reaps the children. Every `SIGCHLD` can interrupt the `accept`.
- `EINTR` is not an error, and `perror` on it fills the log with lines that mean a child exited.
- That is why the loops of the last two frames test it before returning -1.

SA_RESTART, and where it does not help

```
struct sigaction sa;
memset(&sa, 0, sizeof sa);
sa.sa_handler = reap;
sa.sa_flags   = SA_RESTART;      /* restart what can be restarted */
sigemptyset(&sa.sa_mask);
sigaction(SIGCHLD, &sa, NULL);
```

- Only `sigaction` can ask for it — one more reason why `signal` is the call you find in old code and do not write.
- With it, the kernel re-enters `accept`, `read` and `write` itself: your `EINTR` branch simply never fires.
- `select`, `poll` and `nanosleep` are never restarted, with or without it: restarting them would restart their timeout as well, and the wait would never end.
- So the `EINTR` branch stays in the code anyway. It costs one line and it is the only portable answer.

The write that kills the server

A client disappears and you send to it anyway. What happens to the server?

It dies. The default disposition of `SIGPIPE` terminates the process, and the first write after the peer's close is enough: one client closing a window takes down everybody's chat.

```
signal(SIGPIPE, SIG_IGN);                    /* once, before the first client */
ssize_t w = send(fd, p, len, MSG_NOSIGNAL);   /* or per call */
if (w < 0 && errno == EPIPE) { close(fd); drop_client(fd); }
```

- Ignoring turns the signal into a value you can test; add the flag to `send_all` and the whole program is covered.
- On the reading side you find out differently, and it is not an error: `recv` returns 0.

The flags of `send` and `recv`

`0`	exactly what `read` and `write` do
`MSG_WAITALL`	wait until all the requested bytes are there
`MSG_PEEK`	read without removing from the receive queue
`MSG_DONTWAIT`	this call only is non-blocking
`MSG_NOSIGNAL`	no `SIGPIPE`: report `EPIPE` instead

- `MSG_DONTWAIT` is the per-call form of what the event loop sets once on the descriptor with `O_NONBLOCK`.
- `MSG_WAITALL` does not replace `recv_all`: it still returns short when the peer closes, when a signal arrives, or when the socket errors.

Giving up on time

```c
struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);

ssize_t n = recv(fd, buf, sizeof buf, 0);
if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK))
    /* two seconds and nothing arrived: retransmit, or drop this client */
```

- The option belongs to the descriptor, so every read on it gives up after the same interval; `SO_SNDTIMEO` is the twin.
- `recv_all` now has a third outcome, and the caller must be able to tell them apart: n bytes, peer gone, timed out.
- This is what answers the client that sends a header and stops, and it is the timer of the retransmission at the end of today.
- The kernel's own idle check, `SO_KEEPALIVE`, starts probing after about two hours: a chat that must notice a dead peer sends its own heartbeat.

A protocol you can type

```
$ nc 127.0.0.1 5000
JOIN alice
OK welcome, alice
MSG hello everybody
HELLO everybody                a typo, on purpose
ERR 400 unknown message type
```

- `nc` opens a socket and wires it to your terminal: what you type goes out, what arrives is printed. A generic client.
- So a text protocol can be exercised before its client exists, and taken apart by hand when the client misbehaves.
- That is why SMTP, HTTP and WHOIS are typeable — and it is exactly what a binary protocol costs you.
- Careful with the line ending: many protocols specify `\r\n`, and `nc` sends only `\n`; `nc -C` sends `\r\n`.

Communication without a connection

UDP

Independent messages, called datagrams, sent from one endpoint to another with no connection and no guarantee that they arrive.

- No handshake and no release: the first datagram is the first packet on the wire, and there is no state to set up or tear down.
- So there is nothing to `accept`: one socket serves every client the server has, and the sender's address comes with each message.
- A datagram arrives whole or not at all — boundaries are preserved, which is the framing of this lecture, for free.
- And nothing else is promised: not delivery, not order, not that it arrives once.
- Worth it when the exchange is one request and one reply, or when late data is useless data — voice, video, a game.

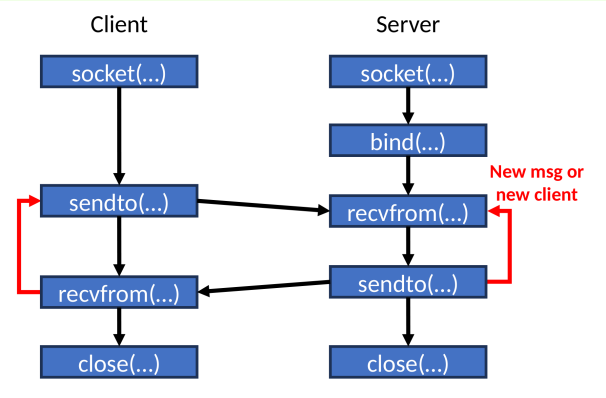
sendto and recvfrom

```
ssize_t sendto(int fd, const void *buf, size_t len, int flags,
               const struct sockaddr *to, socklen_t tolen);

ssize_t recvfrom(int fd, void *buf, size_t len, int flags,
                 struct sockaddr *from, socklen_t *fromlen);
```

- Every send carries the destination: there is no connection holding it for you.
- `recvfrom` fills in who sent it, and that address is the only way the server can reply.
- The return value is the length of one datagram: never a piece of one, never two joined together.
- An error means a local error — too big, no route — and never that the datagram failed to arrive. Nobody knows that yet.
- `fromlen` is value-result: you set it to the size of your structure, the kernel writes back the size it used.

The shape of a UDP server and client



Four calls instead of seven: no `listen`, no `accept`, no second socket. The loop on the right is the whole server, and the one on the left keeps the conversation going.

A UDP echo server

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);
struct sockaddr_in addr = {0};
addr.sin_family      = AF_INET;
addr.sin_port        = htons(5000);
addr.sin_addr.s_addr = htonl(INADDR_ANY);
bind(fd, (struct sockaddr *) &addr, sizeof addr);   /* the client must find us */

for (;;) {
    struct sockaddr_in from;
    socklen_t flen = sizeof from;          /* value-result: set it every time */
    ssize_t n = recvfrom(fd, buf, sizeof buf, 0,
                         (struct sockaddr *) &from, &flen);
    if (n < 0) { if (errno == EINTR) continue; perror("recvfrom"); break; }
    sendto(fd, buf, (size_t) n, 0, (struct sockaddr *) &from, flen);
}
```

There is no per-client descriptor, so there is nothing to `close` and nothing to give to a thread: state per client, if you want it, is yours to keep — in a table, keyed by address and port.

A UDP client

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);    /* no bind: the kernel picks a port */
struct sockaddr_in srv = {0};
srv.sin_family = AF_INET;
srv.sin_port   = htons(5000);
inet_pton(AF_INET, "127.0.0.1", &srv.sin_addr);

sendto(fd, "hello", 5, 0, (struct sockaddr *) &srv, sizeof srv);

ssize_t n = recvfrom(fd, buf, sizeof buf - 1, 0, NULL, NULL);
if (n >= 0) { buf[n] = '\0'; printf("%s\n", buf); }
```

- No `connect`, no handshake: the first `sendto` is the first packet, and it also fixes the client's ephemeral port.
- That port is the address the server replies to, and it exists only because the `sendto` happened.
- `NULL` for the sender means accept a reply from anybody: not only from the server you wrote to.

One `sendto`, one datagram

- One call sends one message and one `recvfrom` receives one: two sends never merge, and one send never splits.
- If your buffer is smaller than the datagram, the rest is discarded and you are not told — unless you ask with `MSG_TRUNC`, which returns the true length.
- The maximum is about 64 KB, but that is the limit of the format, not good advice.
- A datagram too large for the links on the way is cut into pieces below, and one lost piece loses all of it: we will see how, at the network layer.
- In practice keep it small — a few hundred bytes for a chat line — and it travels as one piece.
- Corrupted datagrams are dropped by a checksum underneath, so what your program sees is silence, never garbage.

connect on a datagram socket

```
1 connect(fd, (struct sockaddr *) &srv, sizeof srv);   /* nothing is transmitted */
2
3 send(fd, msg, len, 0);                       /* the destination is recorded now */
4 ssize_t n = recv(fd, buf, sizeof buf, 0); /* and only that peer is delivered */
```

- On a datagram socket `connect` only records the peer: no packet leaves, nothing is negotiated, nobody is told.
- After it you may use `send`, `recv`, `write` and `read`, and datagrams from other addresses are discarded — the cheap answer to the previous frame.
- And it is the only way to be told that nobody is listening on the far port: without it, the message that says so is dropped, because the kernel does not know which socket to give it to.
- Who sends that message, and what else it can say, is a question for the network layer.

What the network does to your datagrams

- Lost: a queue fills up in a router, or the receiver's buffer is full. The datagram is dropped and nobody is told — not the sender, not the receiver.
- Duplicated: a retransmission somewhere below, or a routing loop that resolves itself, and the same message arrives twice.
- Reordered: two datagrams take different paths, and the second one arrives first.
- Corrupted: caught by the checksum and dropped, so for your program it is one more loss.
- The API reports none of it: `sendto` returns success as soon as the datagram leaves, and `recvfrom` just goes on waiting.
- TCP was doing this work all along. If you want it here, you write it.

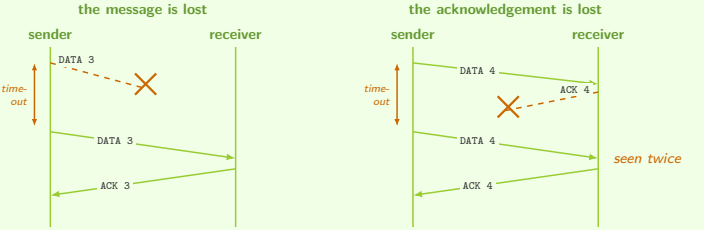
What you have to add

- An acknowledgement: the only way to know a message arrived is that the other side says so. Nothing below will tell you.
- A timeout: the only way to notice a loss is that the acknowledgement does not come within some time you chose.
- A retransmission: when the timer fires, send it again. This is the whole of the recovery.
- A sequence number: because a retransmission that turns out not to have been needed arrives as a second copy.
- One message in flight at a time is stop-and-wait: simple, correct, and as slow as one round trip per message.
- We will meet all four again as a general mechanism when we ask how reliable transfer is built on an unreliable channel.

The retransmission nobody needed

The message arrived, and you retransmit anyway. Why?

Because the acknowledgement is a datagram too, and it can be lost. The sender sees the same thing in both cases — nothing — and the receiver gets a copy it cannot tell from a new message.



The same duplicate appears with nothing lost, if the timer is shorter than the round trip: how to choose it is a question for TCP.

Sequence numbers: discard, but acknowledge again

- Every message carries a number, and every acknowledgement carries the number of the message it refers to.
- The receiver keeps the last number it accepted: one it has already seen is a duplicate, and the content is thrown away.
- But it must be acknowledged again. The sender is retransmitting because it heard nothing, and silence would keep it retransmitting until it gives up.
- An acknowledgement without a number is worse than none: it stops the timer of whichever message happens to be in flight.
- With one message in flight a single alternating bit is enough — and a counter costs four bytes and is far easier to read in a dump.

The retransmission loop

```c
for (int tries = 0; tries < MAX_TRIES; tries++) {
    send_frame(fd, &srv, seq, body, len);              /* one sendto */

    for (;;) {                                  /* wait for the right answer */
        ssize_t n = recvfrom(fd, ack, sizeof ack, 0, NULL, NULL);
        if (n < 0 && errno == EINTR) continue;       /* a signal: keep waiting */
        if (n < 0 && errno == EAGAIN) break;      /* the timer fired */
        if (n < 0) return -1;                        /* a real error */
        if (ack_seq(ack) == seq) return 0;             /* acknowledged: done */
        /* an acknowledgement for an older message: ignore it, keep waiting */
    }
}
return -1;                      /* MAX_TRIES datagrams, no answer: tell the user */
```

The timer is `SO_RCVTIMEO` on this socket. A signal must not consume a try, and giving up is part of the protocol: retransmitting for ever is not reliability, it is a stuck program.

Summary

- A protocol fixes message types, syntax, meaning and rules — and the hardest of the four is where a message ends.
- Three answers, and only three: a delimiter, a length in front, or the close of the connection.
- A length in front is worth nothing without loops that read and write exactly n bytes, and a cap on what you believe.
- A server that survives its clients ignores `SIGPIPE`, retries on `EINTR`, and gives up on time.
- UDP keeps message boundaries and promises nothing else: one `sendto` is one datagram, and it may be lost, duplicated or overtaken.
- Reliability over it is four things you write yourself: acknowledgement, timeout, retransmission, sequence number.

Try it: the smallest application protocol there is

What to write

A `whois` client: resolve `whois.iana.org` with `getaddrinfo`, connect to port 43, send the domain name followed by `\r\n`, print everything that comes back, close.

```
$ ./whois unina.it            then the same by hand, to see the protocol
$ nc -C whois.iana.org 43
unina.it
```

Which of the three framings does WHOIS use — and what tells your loop that the answer is finished? Then send a second name on the same connection: what happens, and why was that predictable?

Since 2025 the generic domains (`.com`, `.org`, ...) publish their data through RDAP, JSON over HTTPS. WHOIS survives for many others, and for this exercise.

Iteration 6: a protocol with frames and errors

Where you restart from

The pooled chat. It serves everybody, and a message is still a line of text: no type, no length, and no way for the server to refuse one.

- What changes: the wire format — four bytes of length, one of type, then the body — with `send_all` and `read_frame` replacing every bare `read` and `write`.
- What is new: error replies with a code, a declared maximum length that you refuse before allocating, and a read timeout for the client that stops halfway through a frame.
- What does not change: the pool, the queue and the roster. If you find yourself editing them, you are solving the wrong problem.

```
$ printf "\x00\x00\x00\x05\x02hello" | nc 127.0.0.1 5000
$ printf "\xff\xff\xff\xff\x02"      | nc 127.0.0.1 5000   must be refused
```

Iteration 7: the same chat over datagrams

Where you restart from

Iteration 6, which is correct because TCP is underneath it. Same protocol, same message types, one socket of type `SOCK_DGRAM`.

- What changes: no `accept` and no descriptor per client: the roster is a table of addresses, and a client exists because a datagram arrived from it.
- What you get free: framing: one message is one datagram.
- What is new: a sequence number and an acknowledgement for every message, `SO_RCVTIMEO` as the timer, and a duplicate discarded and acknowledged again.
- Then measure: drop one datagram in five on purpose, and count the copies your receiver sees before the sequence check.

Now set the timer to 50 ms on a path with an 80 ms round trip. How many copies arrive, and how many losses does your sender report?